Published: 30 June 2016
Accepted: 01 April 2016
Revised: 01 April 2016
Received: 01 August 2015

RESEARCH-ARTICLE

Using Cohesion and Coupling for Software Remodularization: Is It Enough?

IVAN CANDELA, University of Molise, Campobasso, CB, Italy

GABRIELE BAVOTA, Free University of Bozen-Bolzano, Bolzano, BZ, Italy

BARBARA RUSSO, Free University of Bozen-Bolzano, Bolzano, BZ, Italy

ROCCO OLIVETO, University of Molise, Campobasso, CB, Italy

Using Cohesion and Coupling for Software Remodularization: Is It Enough?

IVAN CANDELA, University of Molise
GABRIELE BAVOTA and BARBARA RUSSO, Free University of Bozen-Bolzano
ROCCO OLIVETO, University of Molise

Refactoring and, in particular, remodularization operations can be performed to repair the design of a software system and remove the erosion caused by software evolution. Various approaches have been proposed to support developers during the remodularization of a software system. Most of these approaches are based on the underlying assumption that developers pursue an optimal balance between cohesion and coupling when modularizing the classes of their systems. Thus, a remodularization recommender proposes a solution that implicitly provides a (near) optimal balance between such quality attributes. However, there is still no empirical evidence that such a balance is the *desideratum* by developers. This article aims at analyzing both objectively and subjectively the aforementioned phenomenon. Specifically, we present the results of (1) a large study analyzing the modularization quality, in terms of package cohesion and coupling, of 100 open-source systems, and (2) a survey conducted with 29 developers aimed at understanding the driving factors they consider when performing modularization tasks. The results achieved have been used to distill a set of lessons learned that might be considered to design more effective remodularization recommenders.

CCS Concepts: • **Software and its engineering → Software maintenance tools;**

Additional Key Words and Phrases: Remodularization, software quality

ACM Reference Format:
Ivan Candela, Gabriele Bavota, Barbara Russo, and Rocco Oliveto. 2016. Using cohesion and coupling for software remodularization: Is it enough? ACM Trans. Softw. Eng. Methodol. 25, 3, Article 24 (May 2016), 28 pages.
DOI: http://dx.doi.org/10.1145/2928268

1. INTRODUCTION

Software systems are incrementally changed to meet new requirements, fix defects, or optimize quality attributes [Swanson 1976]. Such changes might conflict with requirements or design decisions implemented in earlier iterations. Thus, when implementing a change request, developers are often at a crossroad [Cunningham 1993] regarding (1) implementing it without compromising the original design integrity or (2) implementing it in the most straightforward way by stretching a bit the rules of the existing design if needed (i.e., by introducing *technical debt*). While the first strategy requires a higher effort to be implemented but preserves the quality, the second strategy is less expensive but could negatively impact on the quality of the original design. Due to time constraints and/or the absence of software design documentation, developers could opt

Authors' addresses: I. Candela and R. Oliveto, University of Molise, Department of Bioscience and Territory, C.da Fonte Lappone, 86090 Pesche (IS), Italy; emails: ivankcandela@gmail.com, rocco.oliveto@unimol.it; G. Bavota and B. Russo, Free University of Bozen-Bolzano, Faculty of Computer Science, Piazza Domenica 3, 39100 Bolzano (BZ), Italy; emails: {gabriele.bavota, barbara.russo}@unibz.it.

DOI: http://dx.doi.org/10.1145/2928268

for the second strategy [Eick et al. 2001; Fowler 1999]. Consequently, the continuous changes made during software evolution generally tend to "erode" the original design of the system [Parnas 1994].

An eroded system is usually characterized by a reduction of the cohesiveness of software modules and by the increment of the coupling between them [Lanza and Marinescu 2006; Parnas 1994]. In object-oriented systems, this might manifest in poorly cohesive packages (i.e., packages grouping together unrelated classes) exhibiting several dependencies toward other packages of the system (i.e., high coupling). These characteristics make the software system harder to comprehend, maintain, and, as a consequence, evolve [DeRemer and Kron 1976]. In such a scenario, refactoring and, in particular, remodularization operations are required to repair the software system aiming at improving its design quality [Fowler 1999; Nierstrasz et al. 2003].

Unfortunately, remodularization might be quite expensive to be performed manually [Hall et al. 2014]. For this reason, various automatic approaches aimed at supporting such a task have been proposed in literature (e.g., Maqbool and Babri [2007], Mitchell and Mancoridis [2006], Praditwong et al. [2011], and Wiggerts [1997]). While the underlying algorithm can substantially vary across the proposed methods (ranging from clustering [Anquetil and Lethbridge 1999; Beck and Diehl 2013; Maqbool and Babri 2007; Wiggerts 1997] to formal concept analysis [Siff and Reps 1997], search-based optimization techniques [Mitchell and Mancoridis 2006; Praditwong et al. 2011], and heuristics-based solutions [Bavota et al. 2013, 2014]) the basic idea behind most of these approaches is to (1) group together in a module highly cohesive source code components (e.g., classes) and (2) reduce the coupling between modules.

However, cohesion and coupling are two contrasting goals. For instance, by placing all classes of an object-oriented system into a single package (*flat modularization*), it is possible to obtain a modularization having the lowest possible coupling (i.e., zero coupling) at the expense of very low cohesion (i.e., the package will group together classes implementing heterogeneous responsibilities). Instead, by putting each class in a separate package, it is possible to obtain maximum cohesion (i.e., each package implements exactly one responsibility) at the expense of high coupling between packages. For this reason, the existing modularization recommenders look for a partitioning of code components that implicitly balances cohesion and coupling. Recently, the cohesion versus coupling compromise has been explicitly modeled by defining software remodularization as a multiobjective problem where the two aforementioned contrasting goals and other quality metrics are considered as independent objective functions [Praditwong et al. 2011]. Differently from previous approaches, a multiobjective remodularization recommender is able to produce a set of (near) optimal solutions (*Pareto front*), where each solution provides a (near) optimal specific compromise between different quality aspects, including cohesion and coupling.

Given the underlying assumption (i.e., a solution balancing high cohesion and low coupling is the *desideratum* by developers), the existing remodularization recommenders have been mostly evaluated by comparing the original design and the suggested modularization with the aim of quantifying the quality improvement achieved with the new modularization in terms, for instance, of cohesion and coupling. While other important factors could be taken into account when modularizing a software system (e.g., module's encapsulation, size, complexity, etc.), cohesion and coupling are the most exploited driving principles in the research literature to recommend high-quality modularizations (see, e.g., Mitchell and Mancoridis [2006], Wiggerts [1997], Praditwong et al. [2011], and Bavota et al. [2013]).

Some researchers started questioning the use of cohesion and coupling as "holy principles" considered by practitioners when modularizing their system [Brito e Abreu and Goulao 2001; Beck and Diehl 2011; de Oliveira et al. 2015]. However, their

observations are generally based on quantitative studies performed on a limited number of systems (up to 16). In this article, we perform two studies aimed at objectively (*Study I*) and subjectively (*Study II*) investigating the role played by cohesion and coupling in the modularization of software systems. *Study I* is a large empirical study performed on 100 Java open-source systems aimed at providing an idea of the optimality of the original design of the object systems and analyzing whether their design provides a balance between structural/conceptual cohesion and coupling or tends to privilege one quality attribute over the other. In order to perform this analysis, we compute on such systems a set of modularizations representing specific and optimal compromises between cohesion and coupling. Identifying such a set of solutions in an exhaustive way is impracticable due to the combinatorial nature of the software remodularization problem. Thus, we approximated the set of optimal modularizations using a search-based approach that reduces the solution space by iteratively selecting a subset of solutions that might lead to an optimal solution. In particular, we employ a two-objective approach aimed at maximizing package cohesion and minimizing package coupling. Once we have identified the set of (near) optimal remodularizations, we measure how far, in terms of refactoring operations, the original modularization of each system is from the solution providing the optimal compromise between cohesion and coupling.

Study II is a survey conducted with 29 developers contributing to a subset of the 100 systems exploited in *Study I*. The goal of this second study is to subjectively analyze the software remodularization phenomenon and identify the driving factors developers consider when performing modularization tasks.

Our studies represent, to the best of our knowledge, the largest quantitative and qualitative investigation into the role played by cohesion and coupling during the modularization of software systems. The achieved results allowed us to corroborate on a larger scale previous literature findings, and indicate the following:

(1) *Study I*: The modularization of open-source systems is generally far from an optimal modularization in terms of both structural and conceptual cohesion/coupling. With very few exceptions, the refactoring effort needed to reach an optimal modularization from the original design is very high. This confirms previous findings/observations [Brito e Abreu and Goulao 2001; Beck and Diehl 2011; de Oliveira et al. 2015], indicating that a modularization obtained by optimizing cohesion and coupling is generally very far from the original module structure of software systems.
(2) *Study II*: 83% of the surveyed developers claimed that other properties besides cohesion and coupling guided the modularization of their systems, showing that recommenders only pursuing high cohesion and low coupling might not be enough to suggest remodularization solutions.

Based on the results achieved, we distilled a set of lessons learned that might be considered to design more effective remodularization tools aiming at facilitating technology transfer from academia to industry.

2. BACKGROUND

This section provides basic notions about the software modularization problem and genetic algorithms, the search-based technique used in our study.

2.1. Software Modularization

Generally speaking, software modularization can be seen as a graph partitioning problem, whose solution is known to be NP-hard [Garey and Johnson 1979]. We introduce the software modularization problem by illustrating how it has been formalized by

Mitchell and Mancoridis in their *Bunch* tool [Mitchell and Mancoridis 2006]. Bunch operates on a system representation called Module Dependency Graph (MDG), a graph $G = (V, E)$ where nodes V are code components and edges E are relations between such components (e.g., method calls). Starting from the MDG (weighted or unweighted), the output of a software module clustering algorithm is represented by a partition of this graph. A good partition of an MDG should be composed of clusters of nodes having (1) high dependencies among nodes belonging to the same cluster (i.e., high cohesion) and (2) few dependencies among nodes belonging to different clusters (i.e., low coupling). To capture these two desirable properties of the system decompositions (and thus, to evaluate the modularizations generated by Bunch), Mancoridis et al. [1998] define the Modularization Quality (MQ) metric:

$$MQ = \begin{cases} (\frac{1}{k}\sum_{i=1}^{k} A_i) - (\frac{1}{\frac{k(k-1)}{2}}\sum_{i,j=1}^{k} E_{i,j}) & \text{if } k > 1 \\ A_1 & \text{if } k = 1, \end{cases}$$

where A_i is the Intraconnectivity (i.e., cohesion) of the i^{th} cluster and $E_{i,j}$ is the Interconnectivity (i.e., coupling) between the i^{th} and the j^{th} clusters. The Intraconnectivity is based on the number of intraedges, that is, the relationships (i.e., edges) existing between entities (i.e., nodes) belonging to the same cluster, while the Interconnectivity is captured by the number of interedges, that is, relationships existing between entities belonging to different clusters.

Bunch allows one to solve the software modularization problem using different search-based optimization heuristics, namely, Hill-Climbing, Simulated Annealing, and Genetic Algorithm (GA). The GA-based approach of Bunch [Mitchell and Mancoridis 2006]—named *Gadget*—has been described by Doval et al. [1999].

One of the most critical choices when building techniques and tools to recommend software modularizations is the information to exploit in order to capture relationships between the code components, that is, how to automatically assess if two code components are related to each other and thus should be placed in the same package to increase cohesion and reduce coupling. Most of the techniques existing in the literature exploit *structural information* extracted by statically analyzing the source code to capture different relations between components, such as the number of calls between two entities, variable accesses, or inheritance relations (see, e.g., Anquetil and Lethbridge [1999], Brito e Abreu and Goulao [2001], Mitchell and Mancoridis [2006], Praditwong et al. [2011], Bavota et al. [2012], Abdeen et al. [2013], and Mkaouer et al. [2015]). Such approaches are based on the conjecture that the higher the number of relations between code components, the higher the likelihood that they should be grouped together in the same module in order to obtain a high-quality modularization.

A second option is represented by *dynamic information*, which takes into account call relationships occurring during program execution. This information is rarely used in recommender systems due to the need of collecting execution traces, which are not always easy to extract.

A third option is to exploit *textual information* to capture latent relations between code components by analyzing the code lexicon (i.e., words used in identifiers and comments) using Information Retrieval (IR) techniques [Baeza-Yates and Ribeiro-Neto 1999]. The conjecture is that the higher the textual similarity between code components, the higher the likelihood that the code components implement similar responsibilities. Thus, lexically similar code components should be grouped together to increase cohesion and decrease coupling. This information has been exploited in numerous remodularization recommenders (see, e.g., Kuhn et al. [2007], Scanniello et al. [2010], Corazza et al. [2010b], Bavota et al. [2013, 2014], and Mkaouer et al. [2015]).

Finally, *historical data* can be used to identify cochanging code components and build a modularization aimed at better isolating the change (i.e., at grouping together code components frequently cochanging over time).

In this article, we exploit structural and textual information to assess the optimality of the original design of the object systems in terms of cohesion and coupling (details are provided in Section 3.1.2). We chose to focus on these two types of information since (1) they are the most used in refactoring and remodularization recommenders, (2) they have been shown to be complementary [Marcus et al. 2008], and (3) they are the ones better assessing the developers' perception of software coupling [Bavota et al. 2013] (i.e., better capturing relationships between code components as perceived by developers).

2.2. Genetic Algorithms

Genetic Algorithms [Goldberg 1989] belong to the family of evolutionary algorithms simulating the evolution of species to approximately solve optimization problems. These algorithms iteratively create populations of individuals, representing solutions for a given problem, to search for a solution giving the best approximation of the optimum for the problem under investigation. A fitness function is used to evaluate the goodness of the solutions represented by the individuals, and genetic operators based on selection and reproduction are employed to generate new populations.

The elementary evolutionary process of these algorithms is composed of the following steps:

(1) A random initial population $P(0)$ is generated and a fitness function is used to assign a fitness value to each individual.
(2) Given t the current generation, some individuals of a population $P'(t-1)$ are selected to form the parents and new individuals are created by applying genetic operators: the crossover operator combines two individuals (i.e., parents) to form one or two new individuals, and the mutation operator is used to randomly modify an individual. Then, the individuals that will survive a selection operator is determined according to the individuals' fitness values.
(3) Step (2) is repeated until stopping criteria hold.

At each generation parents have to be selected for crossover and mutation. Thus, the selection operator plays an important role. The most used selectors are roulette wheel, in which each individual's probability of selection is directly proportional to its relative fitness value, and tournament selection, where small subsets of the population are selected randomly and the fittest member of the subset is selected for the next generation.

Finally, the stopping criterion for the evolutionary process is usually based on a maximum number of generations and combined with other criteria to reduce the computation time. For example, the search process can be stopped when there is no improvement in the fitness value for a given number of generations.

When multiple objectives must be optimized, multiobjective optimization is needed. Here, the quality of a generated solution is expressed as a vector $(y_1, y_2, \ldots, y_k)$ in the objective space Y, where k is the number of objectives. In multiobjective optimization problems, it is necessary to exploit the concept of Pareto dominance: an objective vector y_1 is said to dominate another objective vector y_2 ($y_1 \succ y_2$) if no value of y_1 is smaller than the corresponding value of y_2 and at least one value is greater. Using such a definition, it is possible to define a *set* of optimal solutions, that is, solutions not dominated by any other solution. This set of optimal solutions is generally denoted as the *Pareto set*, while the fitness values achieved by such solutions represent the Pareto front.

Multiobjective genetic algorithms can be exploited in case of multiobjective optimization. In our work, we exploit the Nondominating Sorting Genetic Algorithm (NSGA-II) [Deb 2001], instantiated as described in Section 3.1.2.

The interested reader can find further details about search-based optimization in the books by Goldberg [1989] and by Michalewicz and Fogel [2004]. A survey on the applications of search-based optimization to software engineering problems has been recently provided by Bavota et al. [2014b].

3. STUDY I: MINING SOFTWARE REPOSITORIES

The *goal* of the first study is to objectively analyze the quality of the modularization of open-source systems, with the *purpose* of investigating if developers pay attention in defining an optimal modularization in terms of structural and conceptual cohesion and coupling.

3.1. Study Design

In the context of our study, we aim at answering the following research questions:

—**RQ_1:** *How far is the modularization of open-source projects from an optimal modularization in terms of cohesion and coupling?* This research question aims at investigating to what extent the modularization of open-source systems is optimal in terms of structural and conceptual package cohesion and coupling. Specifically, we analyze how "far" the original modularization of a given system M_S from an optimal modularization M_O is, in terms of the refactoring effort needed to convert M_S in M_O.

—**RQ_2:** *Do open-source developers seek a fair compromise between cohesion and coupling when defining their modularization?* The second research question aims at understanding if open-source developers give priority to cohesion over coupling (or vice versa) when modularizing their system. Given a system S, we analyze the gain (loss) in cohesion and coupling of its current modularization as compared to an optimal modularization representing a fair compromise between cohesion and coupling (i.e., a modularization does not favoring cohesion over coupling or vice versa).

3.1.1. Context Selection. The *context* of the study consists of 100 open-source projects having a size ranging between 14 and 3,062 classes (two and 401 KLOC). The object systems have been randomly selected from the Maven repository[1] among the most popular ones,[2] as will be detailed in Section 3.1.2, and they include popular open-source systems like `Apache Tomcat`, `Struts`, and `Lucene`. The complete list of mined projects is available in our replication package [Candela et al. 2015].

3.1.2. Data Extraction. This subsection describes the data extraction process we followed to answer our research questions.

Collecting the Object Systems. We first collected the 100 releases used in our study by using a crawler that automatically downloads the jar of the object systems from the Maven repository. The crawler takes as input the number n of systems to download (e.g., 100) and randomly selects and downloads a set of n systems (one release each, e.g., 100 releases from 100 different systems) from the 1,000 most popular systems stored in the Maven repository. Our choice of collecting jar files was driven by the need for obtaining the compiled source code of the analyzed software releases where existing tools can be applied to extract class dependencies. Such dependencies (i.e., structural information) are used, as detailed in Section 3.1.2, to measure the structural cohesion

[1] http://mvnrepository.com.

[2] Identified on the basis of the number of client projects using them.

and coupling of the systems' modularization. We exploited the Dependency Finder[3] to extract the dependencies, in terms of method calls, existing between the classes of each release, and we used this information to build an unweighted MDG (see Section 2) representing the classes and their dependencies. While we are aware that other types of dependencies between classes exist (e.g., inheritance), we focus on method calls since they are widely used in remodularization/refactoring recommenders [Bavota et al. 2014].

In addition, we manually downloaded the source code of the collected jar files (i.e., of each software release). This was needed to measure the Conceptual Coupling Between Classes (CCBC) [Poshyvanyk and Marcus 2006], used to compute the conceptual cohesion and coupling of the systems' modularization (as detailed in Section 3.1.2). CCBC is based on the textual information (terms) present in code comments and identifiers. Two classes are conceptually related if their (domain) semantics are similar (i.e., their terms indicate similar responsibilities). CCBC has been shown to capture a new dimension of coupling, which is not captured by the conventional structural metrics. The definition of CCBC requires the introduction of two other measures [Poshyvanyk and Marcus 2006]: the Conceptual Coupling Between Methods (CCM) and the Conceptual Coupling Between a Method and a Class (CCMC). To measure CCM, Latent Semantic Indexing (LSI) [Deerwester et al. 1990] is used to represent each method as a real-valued vector that spans a space defined by the terms extracted from the code. The conceptual coupling between two methods m_k and m_j is then calculated as the cosine of the angle between their corresponding vectors:

$$CCM(m_k, m_j) = \frac{\overrightarrow{m_k} \times \overrightarrow{m_j}}{||\overrightarrow{m_k}|| \times ||\overrightarrow{m_j}||},$$

where $\overrightarrow{m_k}$ and $\overrightarrow{m_j}$ are the vectors corresponding to the methods m_k and m_j, respectively, and $||\overrightarrow{x}||$ represents the Euclidian norm of the vector x [Baeza-Yates and Ribeiro-Neto 1999]. Thus, the higher the value of CCM, the higher the textual similarity between two methods.

As regards the CCMC, given two distinct ($c_i \neq c_j$) classes of a system and a method m_k in c_i, it is measured as the average of the conceptual coupling between method m_k and all the methods from class c_j:

$$CCMC(m_k, c_j) = \frac{\sum_{q=1}^{t} CCM(m_k, m_{jq})}{t},$$

where t is the number of methods in c_j.

Finally, the conceptual coupling between two classes c_k and c_j is defined as

$$CCBC(c_k, c_j) : \frac{\sum_{l=1}^{r} CCMC(m_{kl}, c_j)}{r},$$

which is the average of the coupling between all unordered pairs of methods from class c_k and class c_j. The CCBC is defined in $[0 \ldots 1]$.

Note that, before applying LSI to measure the CCM, we preprocessed the text in each class by using an English stop-words list[4] to remove irrelevant terms, and by applying the Porter stemmer [Porter 1980] to reduce all words to their root.

Measuring Cohesion and Coupling. Once the MDG for a release r_i and the CCBC between all pairs of r_i's classes were computed, we measured the structural and conceptual cohesion and coupling of the modularization of r_i. The structural cohesion was

[3]http://depfind.sourceforge.net.

[4]https://code.google.com/p/stop-words/.

measured by using the lack of structural cohesion (*LStrCoh*) measure. The *LStrCoh* of the modularization r_i is computed as

$$LStrCoh(M_{r_i}) = \frac{\sum_{j=1}^{m} LCOC_{P_j}}{m},$$

where m is the number of packages in r_i, and $LCOC_{P_j}$ is the Lack of Cohesion of Classes for the j^{th} package (P_j) measured as the number of pairs of classes in P_j not having any dependency between them. The *LCOC* metric mirrors the Lack of Cohesion of Methods (LCOM) metric [Chidamber and Kemerer 1994], shifting the focus from class cohesion (LCOM) to package cohesion (*LCOC*). Note that *LCOC* and *LStrCoh* are inverse cohesion measures; the higher *LCOC* (*LStrCoh*) is, the lower the package (modularization) cohesion.

As for the structural coupling (*StrCoup*) of r_i, it is computed as

$$StrCoup(M_{r_i}) = \frac{\sum_{j=1}^{m} FanOut_{P_j}}{m},$$

where $FanOut_{P_j}$ is the number of classes outside P_j referenced by classes in P_j. Note that the higher the value of *StrCoup* is, the higher the overall coupling of the modularization.

Concerning the conceptual cohesion, we exploit the lack of conceptual cohesion (*LConCoh*) measure, computed, for a given r_i's modularization, as

$$LConCoh(M_{r_i}) = \frac{\sum_{j=1}^{m} 1 - ConCoh_{P_j}}{m},$$

where $ConCoh_{P_j}$ is the average *CCBC* between all pairs of classes in P_j ($ConCoh_{P_j} = 1$ if P_j only contains one class). Note that (1) $ConCoh_{P_j}$, as the average of *CCBC* values, is defined in $[0 \ldots 1]$; (2) the higher $ConCoh_{P_j}$ is, the more cohesive P_j; (3) we opted for an inverse cohesion metric (by putting the "1-" in the formula) for the sake of consistency with the *LStrCoh* metric (i.e., the higher the *LConCoh* is, the lower the modularization cohesion).

Finally, we compute the conceptual coupling (*ConCoup*) of r_i as

$$ConCoup(M_{r_i}) = \frac{\sum_{j=1}^{m} CCBCout_{P_j}}{m},$$

where $CCBCout_{P_j}$ is the sum of the *CCBC* between classes in P_j and classes outside P_j. The higher *ConCoup* is, the higher the modularization coupling.

Generating the Set of Near-Optimal Modularizations. To answer our research questions, we needed to compute the set of optimal modularizations for each release under analysis in terms of cohesion and coupling. Finding the partition of classes in the release that maximizes cohesion and minimizes coupling is a combinatorial multiobjective problem. Previous work addressed this problem by looking for an approximation of the optimal solution via search-based approaches. In the context of our study, we exploited a multiobjective genetic algorithm implemented as a Nondominating Sorting Genetic Algorithm [Deb 2001]. We decided to use NSGA-II since it is one of the most effective and efficient genetic algorithms proposed in the literature [Deb et al. 2002] and it has been previously used in the context of search-based software remodularization [Praditwong et al. 2011]. It is worth noting that the goal of our study is not to identify the most accurate algorithm for software remodularization. The GA is just a way to identify a set of different near-optimal modularizations aiming at analyzing how far from these solutions the original modularization of the analyzed software system is.

The exploited GA has the following characteristics:

—**Solution Representation.** Given a software release composed of n classes, the chromosome is represented as an n-sized integer array, where the value $1 \leq v \leq n$ of the i^{th} element indicates the cluster (package) where the i^{th} class is assigned. A solution with the same value (whatever it is) for all elements means that all classes are placed in the same cluster, while a solution with all possible values (from 1 to n) means that each cluster is composed of one class only.

—**Operators.** The crossover operator is a one-point crossover, while the mutation operator randomly identifies a gene (i.e., a position in the array) and modifies it by assigning to it a random value $1 \leq v \leq n$. This means that the corresponding class is moved to cluster v. The selection operator is the roulette-wheel selection.

—**Fitness Functions.** We implemented two variants of the algorithm aimed at optimizing different fitness functions. The first variant aims at identifying near-optimal modularizations in terms of *structural* cohesion and coupling. This is done by minimizing both *LStrCoh* and *StrCoup*. The second variant targets instead the identification of near-optimal modularizations in terms of *conceptual* cohesion and coupling by minimizing *LConCoh* and *ConCoup* (see Section 3.1.2).

We calibrated the parameters of the GA similarly to Praditwong et al. [2011] and Bavota et al. [2012]. Specifically, we used a population size of n individuals for systems having $n > 150$ classes and a population of $2 \times n$ for smaller systems. Praditwong et al. [2011] used a population size of $10 \times n$ instead, because the bigger system used in their study was considerably smaller than the one used in our study (i.e., `Apache Ant` with 3,062 classes). Similarly, we considered a maximum number of generations equal to $20 \times n$ for systems having $n > 150$ components and equal to $50 \times n$ for smaller systems. The crossover probability was set to 0.8, while the mutation rate was set to $0.004 \times log_2(n)$ (as also done by Praditwong et al. [2011] and Bavota et al. [2012]). One parameter also used by genetic algorithms supporting software remodularization is the maximum number of clusters (packages) to extract. Defining the maximum number of clusters a priori is not a trivial task. Such a number highly depends on the system under analysis. Based on our past experience [Bavota et al. 2012], we set the maximum number of clusters to $n/2$, where n is the number of classes in the system release to be remodularized.

In order to mitigate the influence of the GA randomness when defining the set of optimal modularizations, we repeated the process 30 times. This means that we achieved 30 different Pareto fronts (i.e., set of near-optimal modularizations) for each release and for each variant of our algorithm (i.e., 30 Pareto fronts reporting near-optimal modularizations in terms of structural cohesion/coupling and 30 in terms of conceptual cohesion/coupling). Then, we constructed a hybrid frontier by combining the best parts of the different frontiers achieved in all the runs and considering only the solutions that are not dominated by the combined frontier. We considered the achieved frontier as the set of near-optimal modularizations for each release of the object systems. For the sake of brevity, from now on we refer to such solutions simply as "optimal" solutions. Clearly, for each of such solutions, the values of *LStrCoh* and *StrCoup* (*LConCoh* and *ConCoup*) are known (as part of the fitness function), and thus can be compared with the original modularizations.

The data extraction process took ∼60 days on a workstation having a quad-core 2.4Ghz CPU and 8Gb of RAM.

Data Analysis. We represented all the analyzed modularizations in a Cartesian plane having *LStrCoh* (or *LConCoh*, depending on the version of the algorithm) on the x-axis and *StrCoup* (or *ConCoup*) on the y-axis (see Figure 1). Using such a representation of the modularizations, we plotted (1) all the points representing the optimal

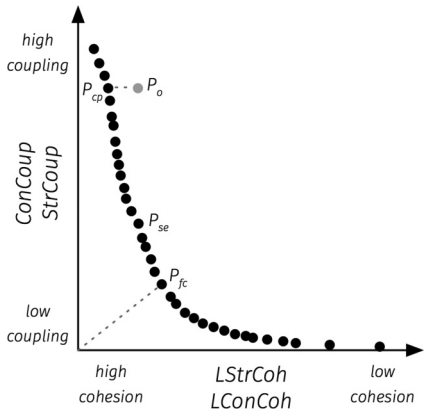

Fig. 1. Points identified in the data analysis process.

modularizations of r_i (i.e., all the solutions in the Pareto front generated by the GA, black points in Figure 1) and (2) a point P_o representing the original modularization of r_i (i.e., the one defined by r_i's developers, the gray point in Figure 1). Then, to answer $\mathbf{RQ_1}$, we identified two points (i.e., two modularizations) in the Pareto front, namely, P_{cp} and P_{se}. The former is the point in the Pareto front (i.e., optimal modularization) closest to the point representing the original modularization of r_i (P_o). We identify it as the point in the Pareto front having the minimum two-dimensional Euclidean distance from P_o. P_{cp} represents the optimal modularization closest to the original one in terms of the cohesion versus coupling compromise sought by r_i's developers. For instance, looking at Figure 1, it is clear that P_o seeks for high cohesion at the expense of high coupling. P_{cp} is the optimal modularization maintaining such compromise: high cohesion at the expense of high coupling.

The other point (P_{se}) is the one in the Pareto front closest to P_o in terms of the effort required to convert P_o into P_{se}. In other words, P_{se} represents the optimal modularization requiring the smallest effort to convert the original modularization into an optimal one. To identify such a point, we measure the MoJo eFfectiveness Measure (MoJoFM) [Wen and Tzerpos 2004] between the original r_i's modularization and that of each modularization in the Pareto front. The MoJoFM is a normalized variant of the MoJo distance and it is computed as follows:

$$MoJoFM(A, B) = 100 - \left(\frac{mno(A, B)}{max(mno(\forall E_A, B))} \times 100 \right),$$

where $mno(A, B)$ is the minimum number of *Move* or *Join* operations one needs to perform in order to transform a partition A into a partition B, and $max(mno(\forall\ E_A, B)$ is the maximum possible distance of any partition A from partition B. *MoJoFM* returns 0 if partition A is the farthest partition away from B; it returns 100 if A is exactly equal to B. We identify P_{se} as the point in the Pareto front representing the modularization having the highest MoJoFM from the modularization represented by P_o.

Once P_o, P_{cp}, and P_{se} are identified, we answer $\mathbf{RQ_1}$ by reporting descriptive statistics of the *MoJoFM* between P_o and P_{cp} (i.e., how far the original modularization is from the optimal one closest to the coupling vs. cohesion compromise sought by the developers in P_o) and between P_o and P_{se} (i.e., how far the original modularization is from the optimal one reachable with the smallest effort) for the analyzed open-source projects. Note that by "far," we really mean the refactoring effort required to convert a modularization into another. Indeed, a recent study by Hall et al. [2014] showed

that the *MoJoFM* between two modularizations A and B strongly correlates with the amount of lines of code a maintainer needs to change in order to convert A into B. We discuss the results separately for the Pareto fronts obtained by optimizing structural and conceptual metrics (see Section 3.1.2). In particular, we use the following notation to distinguish between points extracted from the two Pareto fronts:

—PST_o, PST_{cp}, and PST_{se} are the P_o, P_{cp}, and P_{se} related to the structural metrics optimization, respectively.
—PCO_o, PCO_{cp}, and PCO_{se} are the P_o, P_{cp}, and P_{se} related to the conceptual metrics optimization, respectively.

Concerning $\mathbf{RQ_2}$, we identified on the Pareto front the point P_{fc} (i.e., modularization) representing the fairest compromise between cohesion and coupling among the set of identified solutions. Such a point is computed as the one among the optimal solutions having the lowest Euclidean distance from the origin of the Cartesian coordinate system (see Figure 1). Since the modularization represented by P_{fc} is the one having the fairest optimal compromise between cohesion and coupling (i.e., it does not favor cohesion over coupling or vice versa), knowing the coordinates (i.e., $LStrCoh, StrCoup$ or $LConCoh, ConCoup$) of P_o and P_{fc}, we can compute how much P_o privileges cohesion over coupling (or vice versa) by computing the percentage difference between $LStrCoh(P_{fc})$ ($LConCoh(P_{fc})$) and $LStrCoh(P_o)$ ($LConCoh(P_o)$) as

$$LStrCoh\%_{P_o} = \frac{LStrCoh(P_{fc}) - LStrCoh(P_o)}{LStrCoh(P_{fc})}$$

and between $StrCoup(P_{fc})$ and $StrCoup(P_o)$ (as done for $LStrCoh$). The same formulas also apply for the conceptual metrics.

A modularization P_o having a positive $LStrCoh\%$ ($LConCoh\%$) has a higher structural (conceptual) cohesion than the modularization P_{fc}, while a modularization P_o having a positive $StrCoup\%$ (ConCoup%) has a lower structural (conceptual) coupling than the modularization P_{fc}. Note that, assuming P_{fc} as the optimal fair compromise between cohesion and coupling, P_o can have a better cohesion **or** a better coupling than P_{fc}, but it cannot be better from both points of view (i.e., it must privilege cohesion over coupling or vice versa). Since we are using a search-based algorithm to identify the near-optimal modularizations, it is theoretically possible to have a P_o better in terms of both cohesion and coupling than P_{fc}. However, this never happened in our study. Note also that it is instead possible for P_o to have a worse cohesion **and** coupling than P_{fc}. To answer $\mathbf{RQ_2}$, we report descriptive statistics of the number of projects privileging cohesion over coupling (and vice versa) as well as the number of projects exhibiting worse values for both cohesion and coupling with respect to P_{fc}. Also, in this case, results are discussed separately for the Pareto fronts obtained by optimizing structural and conceptual metrics. We use PST_{fc} to refer to the P_{fc} resulting from the structural optimization and PCO_{fc} for the one resulting from the conceptual optimization.

3.1.3. Replication Package. The dataset used in our study is publicly available [Candela et al. 2015]. Specifically, we provide the *R* scripts and working datasets used to produce the plots and tables presented in the article.

3.2. Analysis of the Results

This section discusses the achieved results with the aim of answering the formulated research questions.

3.2.1. How Far Is the Modularization of Open-Source Projects from an Optimal Modularization in Terms of Cohesion and Coupling? Table I reports size attributes and, in particular, the

Table I. Characteristics of P_o, PST_{cp}, PST_{se}, PCO_{cp}, and PCO_{se} (the Reported Package Size (Pack. Size) Is in Terms of Number of Classes)

P_o		PST_{cp}		PST_{se}		PCO_{cp}		PCO_{se}	
#Pack.	Pack. Size	#Pack.	Pack. Size	#Pack.	Pack. Size	#Pack.	Pack. Size	#Pack.	Pack. Size
35	16	71	8	64	9	65	9	62	9

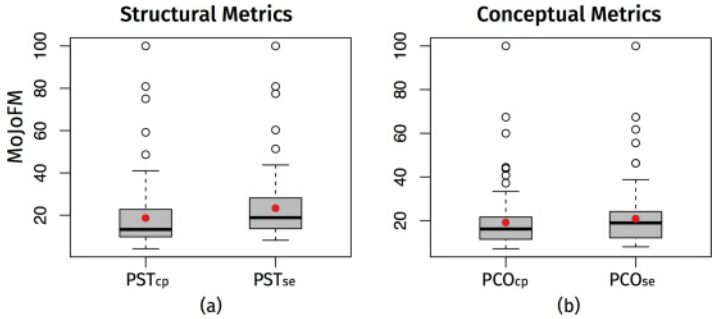

Fig. 2. $\mathbf{RQ_1}$: MoJoFM between P_o and P_{cp}—left side of subfigures (a) and (b)—and P_o and P_{se}—right side of subfigures (a) and (b)—computed on Pareto fronts obtained by optimizing structural (a) and conceptual (b) metrics.

average number of packages and the average number of classes per packages, obtained on the 100 systems for the points P_o, PST_{cp}, PST_{se}, PCO_{cp}, and PCO_{se}. Also, Figure 2 reports the boxplots of the MoJoFM between P_o and P_{cp} and between P_o and P_{se} when analyzing (1) the Pareto fronts obtained by optimizing structural metrics (Figure 2(a)) and (2) those obtained when optimizing conceptual metrics (Figure 2(b)). Remember that the higher the MoJoFM is between two modularizations, the smaller the effort needed to convert one into the other. The analysis of the distributions leads to a very clear conclusion: *the modularization of open-source systems is very far from optimal in terms of structural and conceptual cohesion and coupling*. Indeed, as for the modularizations obtained by optimizing structural metrics (Figure 2(a)), the average MoJoFM between the original design P_o and the optimal modularization closest to P_o in terms of the structural cohesion versus coupling compromise (i.e., PST_{cp}) is 18.7 (median = 13.3), indicating a very high effort needed to convert P_o in PST_{cp}. Also, when looking for the optimal solution requiring the smallest effort to be reached from P_o (i.e., PST_{se}), the MoJoFM values (mean = 23.4, median = 18.9) still indicate how far the original and the optimal modularizations are. Similar observations hold when considering modularizations resulting from the optimization of conceptual metrics (Figure 2(b)). Here, the average MoJoFM between P_o and PCO_{cp} is 19.2 (median = 16.2), while the one between P_o and PCO_{se} is, on average, 20.9 (median = 18.9).

As shown by Hall et al. [2014], the "MoJoFM can provide a relative estimate for the refactoring cost between two competing solutions." This means that, in general, converting the original design of the analyzed object systems into an optimal design (in terms of structural/conceptual cohesion and coupling) requires very expensive refactoring operations. While modern IDEs might provide good support in performing such refactorings, there is also a second drawback in completely changing the modularization of a software system: developers would experience a loss of knowledge about where classes are located in the system (i.e., their "mental model" about the software modularization should be mostly "reconstructed").

Looking at Figure 2, it is clear, however, that there are some exceptions (i.e., outliers). In particular, `ActiveMQ::RA` exhibits an optimal design in terms of both structural and conceptual cohesion/coupling, with a MoJoFM = 100 with respect to both PST_{cp} (PCO_{cp})

and PST_{se} (PCO_{se}), represented by the same point for this system. We looked inside such a system, observing that it contains 37 classes all grouped inside the same package. Such a modularization (flat) is one of the two extremes of the optimal modularizations existing for every software system. Indeed, by grouping together all classes in a single package, it is possible to obtain a modularization having the *lowest possible coupling* (i.e., zero structural and conceptual coupling) at the expense, of course, of a very low cohesion. Thus, while such a modularization is optimal, it represents one of those cases where packages are simply not used to partition classes to isolate responsibilities and, hopefully, changes.

A less trivial and interesting example is represented by `Abdera Core`, composed of 127 classes. In this case, the MoJoFM between P_o and PST_{se} is 77, showing some degree of similarity between the two modularizations. Interestingly, the original design exhibits $LStrCoh = 233$ and $StrCoup = 27$, while PST_{se} has $LStrCoh = 197$ and $StrCoup = 3$. Thus, with a reasonable refactoring effort, it is possible for such a system to achieve a strong improvement of structural coupling (-93%) together with a better value for structural cohesion too ($+15\%$). The `Abdera Core` original design is also not far from an optimal modularization in terms of *conceptual* cohesion and coupling, with a MoJoFM $= 62$ between P_o and PCO_{se}. While Hall et al. [2014] highlighted that the remodularizations recommended by modern automatic tools are generally too expensive from a refactoring point of view, cases like the one of `Abdera Core` suggests that "smarter" recommenders, like the one proposed by Mkaouer et al. [2015], could be able to identify optimal modularizations that are relatively easy to reach from the original design with an acceptable refactoring cost. In these recommenders, the cost needed to reach a modularization starting from the original design is one of the objectives exploited to evaluate the quality of a generated remodularization.

Finally, it is interesting to note that, in general, an original design close to an optimal *structural* modularization is also quite close to an optimal *conceptual* modularization (and vice versa). To statistically verify this trend, we computed the Spearman rank correlation [Zar 1972] between the MoJoFM(P_o,PST_{se}) and the MoJoFM(P_o,PCO_{se}). We obtained a $\rho = 0.52$, indicating a strong positive correlation [Cohen 1988] (i.e., the higher the MoJoFM is between P_o and PST_{se}, the higher the MoJoFM between P_o and PCO_{se}).

> *Summary for* RQ_1. The achieved results highlight that the modularization of open-source systems is generally far from an optimal modularization in terms of both structural and conceptual cohesion/coupling. With very few exceptions, the refactoring effort needed to reach an optimal modularization from the original design as assessed by the MoJoFM is high.

3.2.2. Do Open-Source Developers Seek a Fair Compromise Between Cohesion and Coupling When Defining Their Modularization? When looking at the modularizations obtained by optimizing structural metrics, 57% of the 100 object systems privilege cohesion, 9% coupling, and 34% are worse in terms of both structural cohesion and coupling as compared to PST_{fc}. Thus, it seems that, in general, open-source developers tend to give priority to structural package cohesion over structural coupling when defining the modularization of their systems. To further investigate such a result, we looked inside the package structure of the object systems privileging structural cohesion, trying to understand the reasons/principles guiding the definition of their modularizations. We observed that in many cases, while developers group together related classes (increasing package cohesion), they also tend to clearly separate the different architectural layers composing the system, for example, separate classes belonging to the presentation layer (i.e., GUIs) from those parts of the application logic layer. This means that a class implementing a GUI for a system's feature F_x and one other managing the

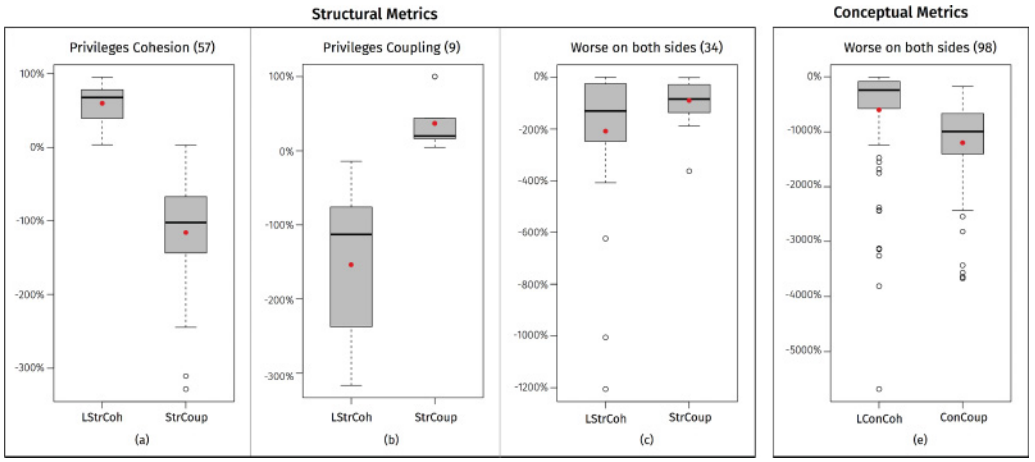

Fig. 3. $\mathbf{RQ_2}$: Percentage difference in cohesion/coupling values for systems *privileging cohesion*, *privileging coupling*, and having *worse values for both measures* as compared to P_{fc}. The red dot spots the mean.

application logic of F_x are often placed into different packages even if they are quite related and exhibit high levels of coupling.

An example of such a choice is the `Apache log4j` system, a logging library for Java. Part of `log4j` is `LogFactor5`, a Swing-based GUI that provides developers with a logging interface for managing log messages. The root package of `LogFactor5` is *org.apache.log4j.lf5*, containing only 10 classes. Despite the low number of classes, this package exhibits dependencies toward five different packages of `log4j` implementing the application logic of such a system. This clearly contributes to increasing the overall coupling of the system. As well as the genetic algorithm we use to identify (near) optimal modularizations, modern software remodularization tools mostly try to group together coupled classes [Mancoridis et al. 1998; Praditwong et al. 2011; Bavota et al. 2014] totally ignoring the role that such classes play in the system. The next generation of such tools should take into account the architectural choices made by developers by automatically deriving such information (as attempted by Scanniello et al. [2010]) or by asking for their feedback during the modularization process, as suggested by recent works in the field [Bavota et al. 2012; Hall et al. 2012].

Moving to the results obtained in the conceptual metrics optimization, we observed a quite different trend with respect to what was discussed for structural metrics. Indeed, of the 100 subject systems, 98 are worse in terms of both conceptual cohesion and coupling as compared to PCO_{fc}, while only two systems privilege conceptual coupling over cohesion. This result is quite surprising, given previous evidence in the literature indicating the conceptual information as the one better assessing the developers' perception of software coupling [Bavota et al. 2013]. In other words, while one would expect developers to group together classes having a high textual similarity (thus improving conceptual cohesion and coupling), they seem not to be able to get even close to optimal values of such metrics. To better understand the differences between the structural/conceptual cohesion and coupling of P_{fc} with respect to the original modularizations of the subject systems, Figure 3 depicts the following:

—The boxplots of the percentage increment/decrement in structural cohesion and coupling (i.e., *LStrCoh%* and *StrCoup%*) measured as explained in Section 3.1.2 for the three categories of systems (i.e., privileges cohesion, privileges coupling, worse on both sides)—Figures 3(a) to 3(c)

—The boxplots of the percentage decrement in conceptual cohesion and coupling (i.e., *LConCoh%* and *ConCoup%*) for the 98 systems that are worse in terms of both *LConCoh* and *ConCoup*—Figure 3(d).

Remember that higher values of *LStrCoh%* (*LConCoh%*) and *StrCoup%* (*ConCoup%*) correspond to **better** values for cohesion (i.e., higher cohesion) and coupling (i.e., lower coupling). Figure 3(a) shows that systems privileging structural cohesion achieve a quite higher cohesion with respect to PST_{fc} (mean = +60%, median = +68%) accompanied, however, by a much higher coupling (mean = +116%, median = +102%). Thus, such systems tend to promote high levels of structural cohesion at the expense of high structural coupling.

Figure 3(b) reveals instead that systems exhibiting a lower structural coupling as compared to PST_{fc} (mean = −37%, median = −20%) pay a lot in terms of cohesion (mean = −154%, median = −113%). An example of such systems is `Lucene`, a Java search engine library. Its packages mainly group together classes that, while being related in terms of the role they play in the system (e.g., all lexical analyzers are grouped in the *org.apache.lucene.analysis* package), exhibit poor structural relationships between them, thus resulting in low cohesive packages.

Figure 3(c) plots *LStrCoh%* and *StrCoup%* for the set of 34 systems having worse values of both structural cohesion and coupling as compared to PST_{fc}. For instance, the `Apache Commons BCEL` system has a structural package cohesion 12 times lower than PST_{fc} and a coupling 84% higher. Figure 3(d) depicts the boxplots of *LConCoh%* and *ConCoup%* for the 98 systems exhibiting worse values of conceptual cohesion and coupling as compared to PCO_{fc}. The original modularizations show a very strong difference in terms of conceptual cohesion and coupling with respect to PCO_{fc}. For example, `ActiveMQ` exhibits a conceptual cohesion three times lower and a coupling four times higher than PCO_{fc}. Also, the data shown in Figure 3(d) confirm the trend already observed for the structural metrics: developers tend to sacrifice package coupling more than cohesion. Indeed, while these 98 systems exhibit worse values for both conceptual cohesion and coupling with respect to PCO_{fc}, the difference observed in terms of cohesion is much smaller than that measured for coupling (Figure 3(d)).

Finally, it is worth highlighting that the achieved results (e.g., several systems having worse values for both cohesion and coupling) do not imply that developers do not care about the partitioning of the system's classes into packages. Probably, other criteria are considered more important by developers for the system modularization. We consider the investigation of these criteria as a fundamental point to build better recommenders, providing meaningful recommendation to developers. By "meaningful remodularization," we refer to remodularization solutions that are reasonable from the developer's perspective (i.e., developers would be willing to implement such remodularization in their systems). Our second study focuses on gathering such information by surveying software developers.

> *Summary for RQ_2.* Overall, open-source systems tend to give higher priority to cohesion over coupling. Our analysis shows that this is mainly due to the clear separation between the application layers pursued by developers when modularizing their system. Also, the results show that package cohesion and coupling might not be the main factors considered by developers when partitioning the classes of a system.

4. STUDY II: SURVEYING SOFTWARE DEVELOPERS

The *goal* of this study is to complement the results achieved in *Study I* by surveying open-source developers. The *purpose* is to understand how frequently developers deal

Table II. Survey Questionnaire Filled in by the Study Participants

Q1. How important is for you the quality of the modularization in a software system?
Very important
Important
Unimportant
Not important at all
Q2. Did you ever remodularize a system?
Often, with both small adjustments (e.g., a single move class refactoring) and massive refactoring operations
Often, with small adjustments
Sometimes, with both small adjustments and massive refactoring operations
Sometimes, with small adjustments
Rarely, with both small adjustments and massive refactoring operations
Rarely, with small adjustments
Never
Q3. Have you ever used a (semi)automatic tool to remodularize a system?
Yes
No
Q4. If yes, which one? Why did you choose to use this tool? — If no, why?
Open answer
Q5. Which properties do you consider important for a high-quality software modularization?
Open answer
Q6. We analyzed the modularization quality of one of the systems you contributed to in terms of package cohesion and coupling (look at the results here). What are the reasons behind such a result?
Open answer

with software remodularization and which are the driving factors they consider when modularizing the classes of their systems.

4.1. Study Design

We aim at addressing the following research question:

—**RQ_3:** *What are the properties of a high-quality modularization for open-source developers?* Stemming from the results of *Study I*, RQ_3 aims at investigating (1) to what extent open-source developers consider important the modularization quality of a software system and how frequently they perform refactoring operations aimed at remodularizing their systems, and (2) which properties (e.g., high cohesion) they consider important for a high-quality modularization.

4.1.1. Context Selection. As potential participants to this study, we invited developers of the systems considered in *Study I*. To identify them, we mined their email addresses from the systems' change log. We only selected active developers (i.e., those who performed at least one commit in the last 3 months) and automatically removed all duplicated email addresses due to developers who contributed to multiple systems. This resulted in 261 developers to contact. Each developer received an email with instructions on how to participate in our study and a link to the Google form hosting our survey. In the end, we collected 29 responses. Even if this number looks low (i.e., the response rate is 11%), it is higher than the suggested minimum response rate for survey studies (i.e., 10%) [Groves 2009].

4.1.2. Data Collection. We answer our research question by asking participants to fill in the questionnaire reported in Table II. The first four questions aim at gathering information about the consideration that participants give to the modularization quality of their systems (Q1), and how frequently (Q2) and with which tools (Q3-Q4) they

perform remodularization. Note that Q3 and Q4 were only asked to participants who did not select *Never* at Q2. Q5 investigates the properties considered by developers as important indicators of high-quality modularization. Note that we did not provide any predefined choice (e.g., high cohesion, low coupling, etc.) to participants in order to not bias their answers. We gave participants the freedom to list any indicator they consider important. Finally, Q6 aims at providing a rationale for the results achieved in the context of *Study I*. We asked developers to explain the reasons for the specific level of quality (in terms of structural and conceptual package cohesion/coupling) we observed in their systems. In particular, Q6 included a link to the results we achieved, where there was a discussion about the quality of the modularization observed for the specific system the developer contributed to. In particular, we showed an Excel file composed of five sheets:

(1) *README*. This sheet reported instructions on how to interpret the subsequent four sheets. In particular, we summarized the design of *Study I*, explaining the structural and conceptual metrics used to assess the quality of the modularization, and the process performed to compute the PST_{se} and the PCO_{se} points. The most important guideline we provided on interpreting the results was related to the MoJoFM. We explained that a MoJoFM >70 between the original modularization and PST_{se}/PCO_{se} is likely to indicate a high-quality modularization from the structural/conceptual point of view.
(2) *Original modularization*. This sheet reported the original modularization shown as a list of class names (i.e., one class per each row) sorted on the basis of the package they belong to. In other words, all classes belonging to the same package were listed one after the other in the Excel sheet. To facilitate the reading and interpretation of the modularization, cells reporting classes belonging to different packages were filled in with different background colors (e.g., all classes belonging to package P_1 were reported in cells filled in yellow, those belonging to P_2 in cells filled in green, etc.).
(3) PST_{se} *structural modularization*. This sheet reported the PST_{se} structural modularization in the same format described for the original modularization.
(4) PCO_{se} *conceptual modularization*. This sheet reported the PCO_{se} conceptual modularization in the same format described for the original modularization.
(5) *Quality metrics*. This sheet reported three tables with the values for the structural and conceptual cohesion/coupling of the different packages composing the original modularization, PST_{se}, and PCO_{se}, respectively. The tables also reported the overall structural and conceptual cohesion/coupling of the three modularizations.

The developers had 4 weeks to complete the survey, after which one of the authors collected the answers in a spreadsheet.

4.2. Analysis of the Results

Figure 4 reports the answers provided by the 29 participants to the first three questions of our survey (those requiring to select one of the possible provided answers). Seventy-five percent of participants consider the modularization quality important (51%) or very important (24%), while just two developers (8%) do not care at all about it.

Four of the surveyed developers never remodularized a system (14%), while only two (8%) claimed to often remodularize software systems with both small and massive refactoring operations. The answers for the remaining 23 questions are distributed as follows: seven (24%) perform remodularization often, seven (24%) sometimes, and four (14%) rarely, but all of them (62% in total) only implement small adjustments (e.g., move class refactoring); the remaining 16% claimed to perform both small and massive

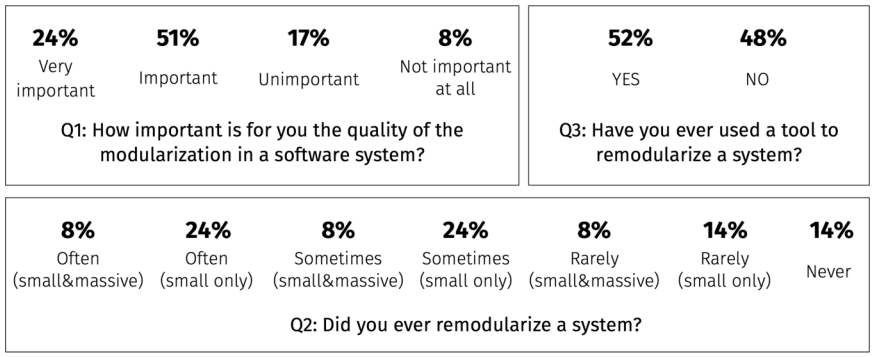

Fig. 4. Results achieved in our survey (multiple-choice questions).

adjustments in terms of system modularization, equally split between the sometimes (8%) and the rarely (8%) frequencies.

Interestingly, 55% of the 27 developers having some experience in software remodularization used tool support. However, their answers to Q4 highlight that they only use the refactoring features provided by their IDEs (specifically, ECLIPSE and NETBEANS) as support to software remodularization: for example, "*I frequently use the refactoring features of Eclipse (move class, etc.) since those are integrated in the IDE, handy, and provide automatic checks for syntactic correctness.*" The reported answer is representative of what the other participants using tool support answered. It is worth noting that none of the surveyed developers declared to use some tool *recommending* how to remodularize a system, like the ones proposed in the research literature [Bavota et al. 2014; Maqbool and Babri 2007; Mitchell and Mancoridis 2006; Praditwong et al. 2011; Wiggerts 1997].

Interesting also are the answers provided by 48% of participants (13) who declared to have never used any tool support while remodularizing their systems. Five of them simply reported to be not aware of any remodularization tool available around (e.g., "*Basically, I do not know any tool for software remodularization (even googled right now)*"). Four developers claimed not feeling the need for tool support given the time they generally spend in software remodularization (e.g., "*The number of operations that I perform does not require any tool support*"). Three participants seem to be not comfortable with tools automatically modifying their code (e.g., "*In general, I am against tools automatically modifying my source code, I prefer to have control over what it is happening in my code*"). Finally, one of them considers remodularization as "*An easy task that can be manually handled.*" All these considerations, together with the results achieved for Q2 indicating that the majority of the participants (62%) have remodularized a software system with just small adjustments, support the recent research direction in the field aimed at defining interactive-based remodularizations [Hall et al. 2012] or approaches to solve specific design problems [Bavota et al. 2014], only requiring focused refactoring operations to be fixed.

Moving to Q5, before looking at the participants' answers, it is worth remembering that for this question, participants were allowed to list more than one property they consider important for a high-quality software modularization. Fifty-two percent (15 participants) consider the clear separation between the application layers as an important property for high-quality software modularization. Note that we consider in this category both answers explicitly referring to the application layers and those referring to the code components' roles (e.g., "*I generally group together classes on the basis of the role they play in the system; this means, for example, that classes used to model entities*

are never mixed with those managing the GUI"). The second most important property from the participants' point of view is the high package cohesion, with 38% of answers (11 participants): for example, "*My only target is to create cohesive modules; if you reach this goal you generally obtain a high quality modularization.*"

Low coupling between packages is considered important by eight (28%) of the surveyed developers, while six (21%) declared grouping together code components that tend to change together, that is, logically coupled [Gall et al. 2003] (e.g., "*I try to group together files that tend to change together. This helps in isolating the change and improves the system's maintainability*"; "*My basic rule is to put together components that I often modify together*"). Such a result confirms that it is worthwhile to consider information about how the code of a software system evolves when supporting software remodularization [Beck and Diehl 2013].

Other developers also provided very interesting insights on the way they partition the system's classes into packages. Two of them claimed that the way software is modularized is somehow dictated by how the different teams work on the different parts of the system (e.g., "*Our policy is to limit the overlap between the different teams working on different parts of the source code, so we group classes based on who leads the different subsystems*"). This suggests the use of "social information" to support software remodularization. To date, such kind of information has been only proposed in a preliminary study as a way to support low-level refactoring operations (e.g., Extract Class) [Bavota et al. 2014a]. One developer explained: "*Since our project is a library, we decided to organize the classes based on the type of services they provide. Thus, all classes not providing any service are grouped in a single package (even if they are logically unrelated).*" This highlights that choices to group classes into specific subsystems without any apparent reason might have a specific rationale, well known to the project's contributors but, for obvious reasons, ignored in the empirical studies we carry out in such a research field as well as by software remodularization recommenders. Finally, one developer highlighted the role played by the GUI components in the remodularization of their system: "*Starting from the GUI, we cluster the classes based on the feature(s) they support.*"

As for Q6, three developers commented on the good results we observed for their systems (having high MoJoFM values for both structural and conceptual P_{se} points), explaining that the reason behind the good cohesion/coupling values we observed for their systems was due to the attention paid in clearly separating responsibilities across the subsystems. As for the remaining 26 developers contributing to the systems for which we observed poor cohesion/coupling values, 11 of them pointed out, in different terms, that they simply do not perform any cohesion/coupling measurement when modularizing their system but just focus on "common sense" to group together classes (e.g., *"The point here is that we do not measure cohesion/coupling to modularize our system, we just use common sense. We do not look at the number of dependencies a class has toward another class to decide if they should be grouped together"*). Four developers referred to the answers they provided in Q5, highlighting that the factors we used to establish the "high quality" of the software modularization are simply not the ones they consider important (e.g., "*As previously said, I consider much more important to group together the code components that often co-change, even if they have no dependencies among them and are not textually similar*"). This highlights once more that structural/conceptual cohesion and coupling, considered by most remodularization recommenders as holy principles for identifying a good solution (see, e.g., Anquetil and Lethbridge [1999], Maqbool and Babri [2007], Mitchell and Mancoridis [2006], Praditwong et al. [2011], Wiggerts [1997], and Bavota et al. [2013, 2014]), should take into account a wider set of factors, possibly dictated by the developers' preferences. Eleven developers were not able to provide any rationale for the observed result, while

one of them pointed out that software modularization is influenced by several factors, and one cannot simply assume a fixed heuristic to assess its quality: "*The quality of a modularization really depends on the type of project, on how frequently does it change, on how many developers work on it, etc. Software modularization is overrated. Often a flat modularization is enough, because the system is simple, or carried out by a very small developers' base.*"

Summary for RQ_3. Most of the surveyed developers (75%) declared to care about the quality of software modularization. Also, 86% performed in the past (with different frequencies) refactoring operations aimed at remodularizing a system. However, only 52% of them exploited tool support during such operations, and no one used a tool recommending how to remodularize a system. Several different properties are considered important to obtain a high-quality modularization, showing that recommenders only pursuing high cohesion and low coupling might not be enough to suggest meaningful remodularization solutions.

5. THREATS TO VALIDITY

This section describes the threats that could affect the validity of both studies presented in Sections 3 and 4. We discuss such threats together since the two studies complement each other providing an objective and subjective analysis, respectively, of the quality of software remodularization in open-source projects.

Construct validity. For *Study I*, the first threat is related to imprecisions in the measurement of package cohesion and coupling. We exploit a state-of-the-art tool (i.e., Dependency Finder) to identify dependencies between the classes of the object systems. Such dependencies represent the basic ingredient to measure package structural cohesion and coupling assessed via the *LStrCoh* and *StrCoup* measures, respectively. Concerning the conceptual metrics, we exploited the well-known CCBC metric to assess the conceptual coupling between pairs of classes and used such information to compute the conceptual cohesion (*LConCoh*) and coupling (*ConCoup*) of packages. Note that besides methods' calls and lexical information extracted from source code comments and identifiers, other sources of information could be exploited to measure coupling between software components (e.g., logical coupling [Gall et al. 1998]). Future work will be devoted to using different coupling measures. Also, it must be clear that our work, at least in its *Study I*, **only sheds light on the cohesion and coupling properties of software modules**, while other desirable properties (e.g., encapsulation, size) [Lanza and Marinescu 2006] might be taken into account and studied in future work. Still, in the context of measuring coupling between modules, it must be taken into account that we considered the coupling of all modules as equally important, while some modules could have high values of coupling by construction [Taube-Schock et al. 2011]. While we are aware of this, the behavior of most remodularization recommenders is exactly the same as the one used in our study: they do not discriminate between coupling coming from different modules when assessing the modularization quality. Finally, when computing the structural dependencies between classes, we opted for an unweighted MDG. It is possible that different results might be achieved by exploiting a weighted MDG.

Another threat is related to the approximations in the identification of the optimal modularizations. As explained in Section 6, software modularization is known to be NP-hard [Garey and Johnson 1979]. For this reason, we looked for an approximation of the optimal solutions by exploiting a multiobjective GA aimed at minimizing both *LStrCoh* and *StrCoup* (*LConCoh* and *ConCoup*). In particular, we consider the set of solutions present in the Pareto front generated by the GA as the set of optimal modularizations

for each release. We mitigated the influence of the GA randomness when defining the set of optimal modularizations by repeating the process 30 times as explained in Section 3.1.2. While the approximation of the optimal modularizations may affect the achieved results (especially those of our $\mathbf{RQ_1}$), algorithms based on exhaustive search were not an option given our desire of considering real software systems (as opposed to toy programs). Also, the fact that none of the original modularizations ever dominated an identified near-optimal modularization makes us confident of the good approximation obtained. The usage of the MoJoFM as a proxy of the refactoring effort could also represent a threat. We acknowledge the existence of several different measures to capture the similarity between two different partitions [Mitchell and Mancoridis 2001]. While different similarity measures might lead to different results, our choice is justified by the fact that, even if the MoJoFM between two partitions A and B is **not** a direct measure of the refactoring effort needed to convert an A into B, it has been shown to strongly correlate with the amount of lines of code a maintainer needs to change to convert A into B [Hall et al. 2014]. To the best of our knowledge, this is the only similarity measure between partitions for which such an empirical evidence exists.

In our survey, we tried not to bias the participants' answers. This was especially needed in the context of Q5 (i.e., *Which properties do you consider important for a high-quality software modularization?*). For this reason, we did not provide a multiple-choice answer to the developers but we used an open answer. In addition, since our Q6 explicitly mentioned cohesion and coupling in its formulation, we allowed developers to read Q6 in our survey only after replying to Q5. This was done by organizing the survey into three different pages: Page 1, containing Q1 through Q4; Page 2, containing Q5; and Page 3, containing Q6.

Internal validity. Such threats typically do not affect exploratory studies like the ones presented in this article. The only case worthy of discussion is the analysis of systems privileging cohesion over coupling or vice versa. Although we observed categories of systems privileging one attribute over the other, we cannot claim that the actual intent of the developers is to remodularize their system based on one attribute (e.g., cohesion). However, such a threat was (partially, due to the number of respondents) mitigated by *Study II*, where we investigated the factors developers consider important when partitioning classes into packages.

Conclusion validity. The analyses performed in the context of *Study I* mainly have an observational nature, and thus we mainly used descriptive statistics and qualitative analysis to support our claims. For *Study II*, the main threat to conclusion validity is the extent to which the set of respondents is representative of the population of developers that worked on the set of applications analyzed in *Study I*. The response rate of our study is 11%, which is above the minimum response rate recommended for survey studies [Groves 2009] (i.e., 10%). Some developers might have not answered our survey due to the high effort required to run the required tasks. This is especially true for developers of large software systems. However, a pool of 29 developers is above the number of participants used in many previous studies investigating other software engineering phenomena [Bavota et al. 2013, 2013; Canfora et al. 2012; Dagenais et al. 2010], where such a number was between 10 and 14.

External validity. We involved in *Study I* a large number of open-source systems. However, due to the need of collecting compiled code for such systems, we randomly selected them among the 1,000 most popular systems hosted in the Maven repository. As a consequence, the selected systems mainly represent very popular Java libraries. Thus, our results might not generalize to systems exploiting specific architectures (e.g., Java EE applications) or written in other languages (e.g., C++). For these reasons, further systems should be analyzed to confirm or contradict our findings.

6. RELATED WORK

In the past, several different techniques have been proposed to support software remodularization. Most of them are based on clustering algorithms [Anquetil and Lethbridge 1999; Corazza et al. 2010a; Kuhn et al. 2007; Maqbool and Babri 2007; Scanniello et al. 2010; Wiggerts 1997], even if a study conducted by Wu et al. [2005] on clustering algorithms shows that they are not ready to be widely adopted on large software systems. Other authors contributed to the software remodularization field via visualization techniques [Ducasse et al. 2007], formal concept analysis [Siff and Reps 1997], heuristics-based solutions [Bavota et al. 2014, 2013], and defining metrics able to capture desirable properties of a modularization [Abdeen et al. 2011]. In this section, we focus attention only on modularization techniques exploiting search-based optimization techniques, that is, the ones used in the context of *Study I*, to identify (near-) optimal modularization solutions for open-source projects. Also, we discuss empirical studies investigating the difference in the assessment of modularization quality from the metrics (e.g., cohesion and coupling) and the developer points of view.

6.1. Search-Based Software Modularization

As explained in Section 2, Mitchell and Mancoridis proposed *Bunch* [Mitchell and Mancoridis 2006], a tool using different search-based optimization heuristics, namely, Hill-Climbing, Simulated Annealing, and GA, to solve the software modularization problem. Harman et al. [2005] studied the robustness of two fitness functions used in the context of single-objective search-based software clustering (i.e., MQ and the EVM introduced by Tucker et al. [2001]). The achieved results show that both metrics are robust, with EVM being the more robust of the two.

Praditwong et al. [2011] introduced two multiobjective formulations of the software remodularization problem, in which several different objectives are represented separately. The two formulations slightly differ for the objectives embedded in the multiobjective function. The first formulation—named Maximizing Cluster Approach (MCA)—has the following objectives: (1) maximizing the sum of intraedges of all clusters, (2) minimizing the sum of interedges of all clusters, (3) maximizing MQ, (4) maximizing the number of clusters, and (5) minimizing the number of isolated clusters (i.e., clusters composed by only one class). The second formulation—named Equal-Size Cluster Approach (ECA)—attempts at producing a modularization containing clusters of roughly equal size. Its objectives are exactly the same as MCA, except for the fifth one (i.e., minimizing the number of isolated clusters) that is replaced with (5) minimizing the difference between the maximum and minimum number of entities in a cluster. The authors compared their algorithms with Bunch. The conducted experimentation provides evidence that the multiobjective approach produces significantly better solutions than the existing single-objective approach though with a higher processing cost. Later on, de Oliveira Barros [2012] analyzed the efficiency of the ECA formulation and concluded that by suppressing MQ from the objective functions, the ECA algorithm can still produce high-quality solutions while using less processing time.

Based on the results achieved by Praditwong et al. [2011] and Bavota et al. [2012] proposed the use of interactive genetic algorithms (IGAs) for software remodularization. IGAs are a variant of GAs where the fitness function is partially or entirely evaluated by a human while the GA evolves. Specifically, Bavota et al. [2012] presented an approach where part of the fitness (capturing aspects such as intramodule, extramodule dependencies, or modularization quality) is automatically evaluated, while the human adds penalties for artifacts that are not where they should be. The reported evaluation shows that IGAs are able to propose remodularizations (1) more meaningful from a

developer's point of view and (2) not worse, and often even better in terms of modularization quality, with respect to those proposed by noninteractive GAs.

The idea of putting the developer in the loop of the software modularization process has also been exploited by Hall et al. [2012] when presenting the SUMO algorithm, an approach allowing the maintainer to refine the results of an automated modularization process. Hall et al. [2014] have also recently shown how a "Big Bang" remodularization (i.e., a complete fully automated reorganization of the system's classes into packages) is not a viable solution. Indeed, even for small systems, the number of changes required to apply it is in the order of thousands of lines of code [Hall et al. 2014]. This further justifies the recent trend for the adoption of interactive-based solutions putting the developer in the loop [Bavota et al. 2012; Hall et al. 2012] or for approaches to solving specific design problems like promiscuous packages [Bavota et al. 2013] or single classes placed in the wrong package [Bavota et al. 2014].

de Oliveira Barros [2013] proposed an incremental search-based technique aimed at remodularizing software systems. The idea is to optimize the software modularization in a sequence of search turns, each one constrained with a maximum number of changes that can be applied to a system. After each search turn, the developer can have a look at the changes required in order to implement the currently recommended modularization and decide whether to continue or not with the next turn. The goal is to take under control the number of changes required to implement a recommended modularization.

Abdeen et al. [2013] exploited NSGA-II to implement a multiobjective remodularization approach aimed at optimizing five objectives: (1) maximizing package cohesion, (2) minimizing package coupling, (3) minimizing package cycles, (4) avoiding Blob packages, and (5) minimizing the modification of the original modularization. Later on, Mkaouer et al. [2015] presented a many-objective software remodularization technique based on NSGA-III. Also, their technique, besides considering structural/conceptual/logical cohesion and coupling as criteria for identifying good modularization solutions, considers the effort required to convert the original modularization into the recommended one as an objective to optimize (in particular, to minimize). Our findings clearly highlighted the need for techniques explicitly considering the "cost" of the remodularization as one of the driving factors when looking for possible remodularization solutions.

6.2. Software Modularization: Quality Metrics Versus Developers' Perception

Brito e Abreu and Goulao [2001] presented the first study questioning the use of cohesion and coupling as "holy principles" considered by practitioners when modularizing their system. In particular, while experimenting with their MOTTO remodularization tool, they observe that it is relatively easy to increase the modularization quality of existing software systems from the cohesion and coupling point of view. This likely indicates that there are other driving forces that guided the original modularization. Their observation is empirically supported by our study.

Beck and Diehl [2011] performed a study on 16 systems to verify which kind of coupling (e.g., structural dependencies, evolutionary coupling, and conceptual similarity) better reflect the way developers modularize their system. Their findings show that none of the investigated forms of coupling reflects the modular structure of the studied systems. With respect to the study by Beck and Diehl [2011], our work includes (1) a much larger quantitative study (100 systems) explicitly modeling the cohesion versus coupling compromise and (2) a complementary qualitative study performed by surveying 29 developers with the aim of understanding which principles they exploit when modularizing their code. Our results quantitatively and qualitatively corroborate what is observed in Beck and Diehl [2011].

de Oliveira et al. [2015] presented an exploratory study on the Apache Ant system aimed at investigating the usage of search-based module clustering as a way to reduce the distance between the current architecture and the design-time one. The authors first observe that the original system design has been lost due to maintenance and evolution activities. Then, they exploit the Hill Climbing search guided by coupling minimization and cohesion maximization to obtain a high-quality modularization possibly close to the original architecture. Their results show that, while the recommended modularization exhibits high quality in terms of cohesion and coupling, it is very complex and hardly acceptable by the original developers. Our quantitative findings, obtained on a much larger set of 100 systems, confirm what has been observed by Barros et al.

Finally, Simons et al. [2015] recently performed a survey in order to verify the validity of the following null hypothesis: *There is no correlation between software metric values and software engineer evaluation of quality for a given software design*. The authors first used quite popular quality metrics (e.g., Direct Class Coupling, Number of Methods) to assess the understandability, reusability, and flexibility of two designs. Then, each participant was asked to express his or her level of agreement to the claim(s): *The shown design is understandable* (the same claim was formulated for reusability and flexibility). The authors observed almost no correlation between the perception of software engineers and the quality metrics, thus not rejecting the formulated null hypothesis. While the goal of our study is different, both studies question in some way the ability of quality metrics to assess "software quality" as perceived by developers.

7. CONCLUSION AND LESSONS LEARNED

Which is the desideratum of developers when performing remodularization tasks? Are cohesion and coupling measures enough to drive software remodularization? These were the driving questions for the empirical studies presented in this article, which sought to objectively and subjectively answer them. Based on the results achieved, we distill the following lessons learned:

(1) *Cohesion and coupling are probably not enough to recommend meaningful remodularization solutions from the developers' point of view*. A high percentage of the analyzed systems exhibit poor values for both structural and conceptual package cohesion and coupling. Also, 83% of the surveyed developers confirmed that other properties besides cohesion and coupling guided the modularization of their systems. Such results reinforce the need for multiobjective software remodularization techniques seeking for the optimization of several different factors and allowing the developer to (1) prioritize specific objectives and (2) provide constraints that the optimization technique should respect while generating the recommended solutions. Indeed, as shown in *Study II*, the modularization of a software system highly depends on its peculiar characteristics. Even an extreme solution, such as flat architecture, might be considered as a valid one in some situations.

(2) *The refactoring effort should be explicitly considered when proposing a modularization*. The results achieved in *Study I* indicated that in some specific cases, quite a few refactoring operations could provide a sensible improvement of the modularization quality. This suggests considering the refactoring effort needed to convert the original design in a recommended modularization as one of the objectives exploited by recommenders. In this way, it is possible to recommend cost-effective software remodularizations. Currently, only very few techniques take such a factor into account while generating modularization solutions [Abdeen et al. 2013; Mkaouer et al. 2015].

(3) *Tools supporting a "Big Bang" remodularization have a limited applicability*. The majority of the surveyed developers claimed that they usually improve the quality of the modularization through small adjustments. Also, there are developers that are still quite skeptical about using tools that automatically change their code. These observations suggest that a recommender proposing a "Big Bang" remodularization of a software system has a quite limited applicability during software evolution. Such an approach might be worthwhile during reverse-engineering tasks, while interactive approaches (see, e.g., Bavota et al. [2012] and Hall et al. [2012]) or recommenders proposing fine-grained refactoring operations (see, e.g., Bavota et al. [2014]) seem to be more adequate during software evolution.
(4) *Some design choices might look suboptimal, but the rationale behind them is simply hidden. Could we mine it?* The developers' answers collected in *Study II* showed interesting cases of modularizations that can be considered of very low quality from the cohesion/coupling point of view but that have a precise rationale behind them. For example, a developer contributing to a software library explained her choice to *group together all classes not providing any service in a single package even if they are logically unrelated*. Modern refactoring/remodularization tools simply ignore this aspect, considering any design choices deviating from specific quality standards (e.g., a large class, a low-cohesive package,etc.) as a design problem. Given the advances made in the Mining Software Repositories (MSR) field, it should be possible to integrate into refactoring/remodularization tools MSR techniques aimed at analyzing developers' discussions in issue trackers and mailing lists with the goal of inferring the rationale behind specific design choices. In this way, some design problems, code smells, or antipatterns could be considered "justified" by the refactoring tool, and thus not fixed.
(5) *The software engineering research community should increase its effort in producing and advertising high-quality refactoring/remodularization tools*. While 55% of the surveyed developers have used refactoring tools to perform remodularization, none of them has ever used a tool developed by the research community (they only used the refactoring features provided by their IDEs). Five of the survey participants explicitly reported to be not aware of any remodularization tool available. These findings clearly highlight the need for the research community of investing more effort in developing and advertising high-quality tools implementing approaches to recommend refactoring/remodularization solutions. Indeed, more often than not very good techniques that underwent an exhaustive and successful empirical validation are never implemented and publicly released as part of a concrete, working tool.

Our research agenda includes the replication of both the studies aiming at corroborating our findings. We plan to enlarge our *Study I* to also include other factors (e.g., modules' size, complexity, etc.) and to investigate if the properties considered important by developers for a high-quality software modularization change at different stages of the software evolution (e.g., when working on the first release vs. when evolving a mature system).

REFERENCES

H. Abdeen, S. Ducasse, and H. A. Sahraoui. 2011. Modularization metrics: Assessing package organization in legacy large object-oriented software. In *Proceedings of the 18th Working Conference on Reverse Engineering (WCRE'11)*. 394–398.

H. Abdeen, H. Sahraoui, O. Shata, N. Anquetil, and S. Ducasse. 2013. Towards automatically improving package structure while respecting original design decisions. In *Proceedings of the 2013 20th Working Conference on Reverse Engineering (WCRE'13)*. 212–221. DOI: http://dx.doi.org/10.1109/WCRE.2013.6671296

N. Anquetil and T. Lethbridge. 1999. Experiments with clustering as a software remodularization method. In *Proceedings of the 7th Working Conference on Reverse Engineering (WCRE'99)*. 235–255.

R. Baeza-Yates and B. Ribeiro-Neto. 1999. *Modern Information Retrieval*. Addison-Wesley.

G. Bavota, F. Carnevale, A. De Lucia, M. Di Penta, and R. Oliveto. 2012. Putting the developer in-the-loop: An interactive GA for software re-modularization. In *Proceedings of the 4th International Symposium on Search Based Software Engineering (SSBSE'12)*. 75–89.

G. Bavota, B. Dit, R. Oliveto, M. Di Penta, D. Poshyvanyk, and A. De Lucia. 2013. An empirical study on the developers' perception of software coupling. In *Proceedings of the 35th International Conference on Software Engineering (ICSE'13)*. IEEE/ACM, 692–701.

G. Bavota, M. Gethers, R. Oliveto, D. Poshyvanyk, and A. De Lucia. 2014. Improving software modularization via automated analysis of latent topics and dependencies. *ACM Transactions on Software Engineering and Methodology* 23, 1 (2014), 4.

G. Bavota, A. De Lucia, A. Marcus, and R. Oliveto. 2013. Using structural and semantic measures to improve software modularization. *Empirical Software Engineering* 18, 5 (2013), 901–932.

G. Bavota, A. De Lucia, A. Marcus, and R. Oliveto. 2014. Recommending refactoring operations in large software systems. In *Recommendation Systems in Software Engineering*. 387–419.

G. Bavota, R. Oliveto, M. Gethers, D. Poshyvanyk, and A. De Lucia. 2013. Methodbook: Recommending move method refactorings via relational topic models. *IEEE Transactions on Software Engineering* 99, PrePrints (2013), 1. DOI:http://dx.doi.org/10.1109/TSE.2013.60

G. Bavota, S. Panichella, N. Tsantalis, M. Di Penta, R. Oliveto, and G. Canfora. 2014a. Recommending refactorings based on team co-maintenance patterns. In *Proceedings of the ACM/IEEE International Conference on Automated Software Engineering (ASE'14)*. 337–342.

G. Bavota, M. Penta, and R. Oliveto. 2014b. Search based software maintenance: Methods and tools. In *Evolving Software Systems*. Springer, Berlin.

F. Beck and S. Diehl. 2011. On the congruence of modularity and code coupling. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE'11)*. 354–364.

F. Beck and S. Diehl. 2013. On the impact of software evolution on software clustering. *Empirical Software Engineering* 18, 5 (2013), 970–1004.

F. Brito e Abreu and M. Goulao. 2001. Coupling and cohesion as modularization drivers: Are we being over-persuaded? In *Proceedings of the 5th European Conference on Software Maintenance and Reengineering, 2001.* 47–57.

I. Candela, G. Bavota, B. Russo, and R. Oliveto. 2015. Using Cohesion and Coupling for Software Remodularization: Is it Enough? – Replication Package. (2015). http://tinyurl.com/pm5ez7m.

G. Canfora, M. Di Penta, R. Oliveto, and S. Panichella. 2012. Who is going to mentor newcomers in open source projects? In *Proceedings of the 20th ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE-20) (SIGSOFT/FSE'12)*. ACM, 44.

S. R. Chidamber and C. F. Kemerer. 1994. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering (TSE)* 20, 6 (June 1994), 476–493.

J. Cohen. 1988. *Statistical Power Analysis for the Behavioral Sciences* (2nd ed.). Lawrence Earlbaum Associates.

A. Corazza, S. Di Martino, and G. Scanniello. 2010a. A probabilistic based approach towards software system clustering. In *Proceedings of the 2010 14th European Conference on Software Maintenance and Reengineering (CSMR'10)*. 88–96.

A. Corazza, S. Di Martino, V. Maggio, and G. Scanniello. 2010b. Investigating the use of lexical information for software system clustering. In *Proceedings of the 14th European Conference on Software Maintenance and Reengineering (CSMR'10)*. 35–44.

W. Cunningham. 1993. The WyCash portfolio management system. *OOPS Messenger* 4, 2 (1993), 29–30. DOI:http://dx.doi.org/10.1145/157710.157715

B. Dagenais, H. Ossher, R. K. E. Bellamy, M. P. Robillard, and J. de Vries. 2010. Moving into a new software project landscape. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 1 (ICSE'10)*. ACM, 275–284.

M. B. de Oliveira, F. de Almeida Farzat, and G. H. Travassos. 2015. Learning from optimization: A case study with apache ant. *Information and Software Technology* 57 (2015), 684–704. DOI:http://dx.doi.org/10.1016/j.infsof.2014.07.015

M. de Oliveira Barros. 2012. An analysis of the effects of composite objectives in multiobjective software module clustering. In *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation (GECCO'12)*. 1205–1212.

M. de Oliveira Barros. 2013. An experimental study on incremental search-based software engineering. In *Search Based Software Engineering*. Lecture Notes in Computer Science, Vol. 8084. Springer, Berlin, 34–49. DOI:http://dx.doi.org/10.1007/978-3-642-39742-4_5

K. Deb. 2001. *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley.

K. Deb, S. Agrawal, A. Pratap, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6, 2 (2002), 182–197. DOI:http://dx.doi.org/10.1109/4235.996017

S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. Harshman. 1990. Indexing by latent semantic analysis. *Journal of the American Society for Information Science* 41, 6 (1990), 391–407.

F. DeRemer and H. H. Kron. 1976. Programming in the large versus programming in the small. *IEEE Transactions on Software Engineering* 2, 2 (1976), 80–86.

D. Doval, S. Mancoridis, and B. S. Mitchell. 1999. Automatic clustering of software systems using a genetic algorithm. In *Proceedings of the Software Technology and Engineering Practice (STEP'99)*. IEEE Computer Society, 73–82.

S. Ducasse, D. Pollet, M. Suen, H. Abdeen, and I. Alloui. 2007. Package surface blueprints: Visually supporting the understanding of package relationships. In *Proceedings of the IEEE International Conference on Software Maintenance, 2007 (ICSM'07)*. 94–103.

S. G. Eick, T. L. Graves, A. F. Karr, J. S. Marron, and A. Mockus. 2001. Does code decay? Assessing the evidence from change management data. *IEEE Transactions on Software Engineering* 27, 1 (Jan. 2001), 1–12.

M. Fowler. 1999. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, Boston, MA.

H. Gall, K. Hajek, and M. Jazayeri. 1998. Detection of logical coupling based on product release history. In *Proceedings of the 14th International Conference on Software Maintenance*. IEEE CS Press, 190–198.

H. Gall, M. Jazayeri, and J. Krajewski. 2003. CVS release history data for detecting logical couplings. In *Proceedings of the 6th International Workshop on Principle of Software Evolution*. IEEE CS Press, 13–23.

M. R. Garey and D. S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman.

D. E. Goldberg. 1989. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley Longman Publishing Co.

R. M. Groves. 2009. *Survey Methodology* (2nd ed.). Wiley.

M. Hall, M. Khojaye, N. Walkinshaw, and P. McMinn. 2014. Establishing the source code disruption caused by automated remodularization tools. In *Proceedings of the 2014 30th IEEE International Conference on Software Maintenance and Evolution (ICSME'14)*.

M. Hall, N. Walkinshaw, and P. McMinn. 2012. Supervised software modularisation. In *Proceedings of the 2012 28th IEEE International Conference on Software Maintenance (ICSM'12)*. 472–481.

M. Harman, S. Swift, and K. Mahdavi. 2005. An empirical study of the robustness of two module clustering fitness functions. In *Proceedings of the 7th Annual Conference on Genetic and Evolutionary Computation (GECCO'05)*. ACM, New York, NY, 1029–1036. DOI:http://dx.doi.org/10.1145/1068009.1068184

A. Kuhn, S. Ducasse, and T. Gîrba. 2007. Semantic clustering: Identifying topics in source code. *Information and Software Technology* 49, 3 (2007), 230–243.

M. Lanza and R. Marinescu. 2006. *Object-Oriented Metrics in Practice: Using Software Metrics to Characterize, Evaluate, and Improve the Design of Object-Oriented Systems*. Springer.

S. Mancoridis, B. S. Mitchell, C. Rorres, Y.-F. Chen, and E. R. Gansner. 1998. Using automatic clustering to produce high-level system organizations of source code. In *Proceedings of the 6th International Workshop on Program Comprehension (IWPC'98)*. 45–52.

O. Maqbool and H. A. Babri. 2007. Hierarchical clustering for software architecture recovery. *IEEE Transactions on Software Engineering* 33, 11 (2007), 759–780.

A. Marcus, D. Poshyvanyk, and R. Ferenc. 2008. Using the conceptual cohesion of classes for fault prediction in object-oriented systems. *IEEE Transactions on Software Engineering* 34, 2 (2008), 287–300.

Z. Michalewicz and D. B. Fogel. 2004. *How to Solve It: Modern Heuristics* (2nd ed.). SV, Berlin, Germany.

B. S. Mitchell and S. Mancoridis. 2001. Comparing the decompositions produced by software clustering algorithms using similarity measurements. In *Proceedings of the IEEE International Conference on Software Maintenance, 2001*. 744–753.

B. S. Mitchell and S. Mancoridis. 2006. On the automatic modularization of software systems using the bunch tool. *IEEE Transactions on Software Engineering* 32, 3 (2006), 193–208.

W. Mkaouer, M. Kessentini, A. Shaout, P. Koligheu, S. Bechikh, K. Deb, and A. Ouni. 2015. Many-objective software remodularization using NSGA-III. *ACM Transactions on Software Engineering Methodology* 24, 3 (May 2015), 17:1–17:45.

O. Nierstrasz, S. Ducasse, and S. Demeyer. 2003. *Object-Oriented Reengineering Patterns*. Morgan Kaufmann Publishers.

D. L. Parnas. 1994. Software aging. In *Proceedings of the 16th International Conference on Software Engineering*. IEEE Computer Society/ACM Press, 279–287.

M. F. Porter. 1980. An algorithm for suffix stripping. *Program* 14, 3 (1980), 130–137.

D. Poshyvanyk and A. Marcus. 2006. The conceptual coupling metrics for object-oriented systems. In *Proceedings of the 22nd IEEE International Conference on Software Maintenance (ICSM'06)*. 469–478.

K. Praditwong, M. Harman, and X. Yao. 2011. Software module clustering as a multi-objective search problem. *IEEE Transactions on Software Engineering* 37, 2 (2011), 264–282.

G. Scanniello, A. D'Amico, C. D'Amico, and T. D'Amico. 2010. Using the Kleinberg algorithm and vector space model for software system clustering. In *Proceedings of the 2010 IEEE 18th International Conference on Program Comprehension (ICPC'10)*. 180–189.

M. Siff and T. W. Reps. 1997. Identifying modules via concept analysis. In *ICSM*. 170–179.

C. Simons, J. Singer, and D. R. White. 2015. Search-based refactoring: Metrics are not enough. In *Search-Based Software Engineering*. Lecture Notes in Computer Science, Vol. 9275. Springer International Publishing, 47–61. DOI:http://dx.doi.org/10.1007/978-3-319-22183-0_4

E. Burton Swanson. 1976. The dimensions of maintenance. In *Proceedings of the 2nd International Conference on Software Engineering.* 492–497.

C. Taube-Schock, R. J. Walker, and I. H. Witten. 2011. Can we avoid high coupling? In *2011 Object-Oriented Programming (ECOOP'11)*, Mira Mezini (Ed.). Lecture Notes in Computer Science, Vol. 6813. Springer Berlin Heidelberg, 204–228.

A. Tucker, S. Swift, and X. Liu. 2001. Variable grouping in multivariate time series via correlation. *IEEE Transactions on Systems, Man, and Cybernetics* 31, 2 (2001), 235245.

Z. Wen and V. Tzerpos. 2004. An effectiveness measure for software clustering algorithms. In *Proceedings of the 12th IEEE International Workshop on Program Comprehension*. 194–203.

T. A. Wiggerts. 1997. Using clustering algorithms in legacy systems remodularization. In *Proceedings of the 4th Working Conference on Reverse Engineering (WCRE'97)*. IEEE Computer Society, 33.

J. Wu, A. E. Hassan, and R. C. Holt. 2005. Comparison of clustering algorithms in the context of software evolution. In *Proceedings of the 21st IEEE International Conference on Software Maintenance, 2005 (ICSM'05)*. 525–535.

J. H. Zar. 1972. Significance testing of the Spearman rank correlation coefficient. *Journal of the American Statistical Association* 67, 339 (1972), 578–580.

Received August 2015; revised April 2016; accepted April 2016